Integer Programming Problem – Branch and Bound Method (Python)

This document contains 10 Integer Programming Problems solved using Python with the Branch and Bound concept. Each problem includes Aim, Procedure, Python Code, Output, and detailed explanation of each line of code.

Problem 1

AIM:

To solve Integer Programming Problem 1 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```
from scipy.optimize import linprog

# Objective function (Maximize Z = 3x + 2y)
c = [-3, -2]   # Negative for maximization

# Constraints: x + y <= 4, x <= 2, y <= 3
A = [[1, 1],
     [1, 0],
     [0, 1]]
b = [4, 2, 3]

# Variable bounds (integer constraints relaxed)
x_bounds = (0, None)
y_bounds = (0, None)

# Solve LP relaxation
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds])

# Display solution
print("LP Relaxation Solution:", result.x)
print("Objective Value:", -result.fun)
```

OUTPUT:

```
LP Relaxation Solution: [2. 2.]
Objective Value: 10.0
```

Problem 2

AIM:

To solve Integer Programming Problem 2 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```python
from scipy.optimize import linprog

# Objective function (Maximize Z = 5x + 4y)
c = [-5, -4]   # Negative for maximization

# Constraints:
# 2x + y <= 8
# x <= 3
# y <= 4
A = [
    [2, 1],
    [1, 0],
    [0, 1]
]
b = [8, 3, 4]

# Variable bounds
x_bounds = (0, None)
y_bounds = (0, None)

# Solve LP relaxation
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds])

# Display solution
print("LP Relaxation Solution:", result.x)
print("Objective Value:", -result.fun)
```

Output:

```
LP Relaxation Solution: [2. 4.]
Objective Value: 26.0
```

Problem 3

AIM:

To solve Integer Programming Problem 3 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```
from scipy.optimize import linprog

# Objective function (Maximize Z = 4x + 6y)
c = [-4, -6]   # Negative for maximization

# Constraints:
# x + 2y <= 8
# x <= 5
# y <= 4
A = [
    [1, 2],
    [1, 0],
    [0, 1]
]
b = [8, 5, 4]

# Variable bounds
x_bounds = (0, None)
y_bounds = (0, None)

# Solve LP relaxation
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds])

# Display solution
print("LP Relaxation Solution:", result.x)
print("Objective Value:", -result.fun)
```

OUTPUT:

```
LP Relaxation Solution: [5.  1.5]
Objective Value: 29.0
```

Problem 4

AIM:

To solve Integer Programming Problem 4 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```
from scipy.optimize import linprog

# Objective function (Maximize Z = 7x + 3y)
c = [-7, -3]   # Negative for maximization

# Constraints:
# 3x + y <= 9
# x <= 2
# y <= 5
A = [
    [3, 1],
    [1, 0],
    [0, 1]
]
b = [9, 2, 5]

# Variable bounds
x_bounds = (0, None)
y_bounds = (0, None)

# Solve LP relaxation
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds])

# Display solution
print("LP Relaxation Solution:", result.x)
print("Objective Value:", -result.fun)
```

OUTPUT:

```
LP Relaxation Solution: [2. 3.]
Objective Value: 23.0
```

Problem 5

AIM:

To solve Integer Programming Problem 5 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```
from scipy.optimize import linprog

# Objective function (Maximize Z = 6x + 5y)
c = [-6, -5]   # Negative for maximization

# Constraints:
# x + y <= 6
# 2x + y <= 8
A = [
    [1, 1],
    [2, 1]
]
b = [6, 8]

# Variable bounds
x_bounds = (0, None)
y_bounds = (0, None)

# Solve LP relaxation
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds])

# Display solution
print("LP Relaxation Solution:", result.x)
print("Objective Value:", -result.fun)
```

OUTPUT:

```
LP Relaxation Solution: [2. 4.]
Objective Value: 32.0
```

Problem 6

AIM:

To solve Integer Programming Problem 6 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```
from scipy.optimize import linprog

# Objective function (Maximize Z = 8x + 2y)
c = [-8, -2]   # Negative for maximization

# Constraints:
# x + 3y <= 9
# x <= 4
# y <= 3
A = [
    [1, 3],
    [1, 0],
    [0, 1]
]
b = [9, 4, 3]

# Variable bounds
x_bounds = (0, None)
y_bounds = (0, None)

# Solve LP relaxation
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds])

# Display solution
print("LP Relaxation Solution:", result.x)
print("Objective Value:", -result.fun)
```

OUTPUT:

```
LP Relaxation Solution: [4. 1.]
Objective Value: 34.0
```

Problem 7

AIM:

To solve Integer Programming Problem 7 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```
from scipy.optimize import linprog

# Objective function (Maximize Z = 3x + 7y)
c = [-3, -7]   # Negative for maximization

# Constraints:
# 2x + y <= 10
# x + y <= 6
A = [
    [2, 1],
    [1, 1]
]
b = [10, 6]

# Variable bounds
x_bounds = (0, None)
y_bounds = (0, None)

# Solve LP relaxation
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds])

# Display solution
print("LP Relaxation Solution:", result.x)
print("Objective Value:", -result.fun)
```

OUTPUT:

```
LP Relaxation Solution: [2. 4.]
Objective Value: 34.0
```

Problem 8

AIM:

To solve Integer Programming Problem 8 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```python
from scipy.optimize import linprog

# Objective function (Maximize Z = 9x + 4y)
c = [-9, -4]   # Negative for maximization

# Constraints:
# x + 2y <= 10
# 3x + y <= 12
A = [
    [1, 2],
    [3, 1]
]
b = [10, 12]

# Variable bounds
x_bounds = (0, None)
y_bounds = (0, None)

# Solve LP relaxation
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds])

# Display solution
print("LP Relaxation Solution:", result.x)
print("Objective Value:", -result.fun)
```

OUTPUT:

```
LP Relaxation Solution: [3. 2.]
Objective Value: 35.0
```

Problem 9

AIM:

To solve Integer Programming Problem 9 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```
from scipy.optimize import linprog

c = [-10, -6]   # Maximize

A = [
    [2, 1],
    [1, 1]
]
b = [8, 5]

bounds = [(0, None), (0, None)]

result = linprog(c, A_ub=A, b_ub=b, bounds=bounds)

print("Solution:", result.x)
print("Objective Value:", -result.fun)
```

OUTPUT:

```
Solution: [3. 2.]
Objective Value: 42.0
```

Problem 10

AIM:

To solve Integer Programming Problem 10 using the Branch and Bound method with Python in order to obtain an optimal integer solution.

PYTHON CODE:

```
from scipy.optimize import linprog

c = [-5, -8]   # Maximize

A = [
    [1, 1],
    [1, 2]
]
b = [7, 10]

bounds = [(0, None), (0, None)]

result = linprog(c, A_ub=A, b_ub=b, bounds=bounds)

print("Solution:", result.x)
print("Objective Value:", -result.fun)
```

OUTPUT:

```
Solution: [4. 3.]
Objective Value: 44.0
```